WAC IoT Unified REST Interface for 3rd party

Version 1.91

Table of Contents

Overview

Supported Devices

This document outlines the REST interface used to interact with the following WAC IoT devices.
Via this interface the device can be configured, adjusted, and controlled.

Device	Minimum Firmware Version(s)
Ventrix	01.04.0173
ColorScaping	01.04.0149
InvisiLED	01.00.0219
WAC Home Gateway	02.01.0001
Gen4 Fan	01.00.0082

License and Usage

Use of the API's included in this guide are governed by the WAC API License Agreement available for download at: https://www.waclighting.com/developer.

Glossary

Below is a list of terms that appear in this document and their associated meanings.

- System – Synonymous with Device.
- Device – Device refers to the product housing the track controller and wireless receiver.
- Fixture – A fixture is a light, sensor, or other product that is part of the Device or System.
- Group – A group is multiple fixtures that can be controlled together as one entity.
- Automation – An automation (sometimes called a scene) is multiple fixtures that are configured to store preset values and can be command to use those presets as a single entity.

Network Configuration

Wi-Fi Network Types: 802.11 b/g/n
Ethernet: 10/100 Mb (802.3)
HTTP Server Port: 80
mDNS Port: 5353

Device Discovery

The device can be discovered on a network using mDNS. The device has a single mDNS service that it advertises, easylink. This service uses the TCP protocol and specifies the port as 443 (corresponding to the HTTPS server port). The service has four text items made up of key value pairs that can be used to get some basic information about the device before communicating via HTTPS.

Hostname: STRUT_XXXXXX – Where XXXXXX is the last six characters of the station MAC address.
Instance name: STRUT_XXXXXX
Service name: STRUT_XXXXXX
Service type: easylink
Service protocol: TCP

Service port: 443

Service text item key-value pairs:

Key	Value
Firmware Ver	Current version of the firmware running on the IOTM.
Protocol	Protocol that the device is using for the HTTPS server. This will have a value of: com.waclighting.strut
Protocol Ver	The current version of the protocol. This is the same as the version of the REST interface.
MAC	The station MAC address of the device.

UDP Multicast Control

All devices ship with a default multicast address/port of 239.87.65.67(WAC in ASCII) Port 52005. Each command will be sent 3 times. It used the JSON format. Each UPD Multicast command contain fixed heard and payload fields.

The fixed header consists of the following fields:

Name	Type	Description
uri	String	**Required.** This is refer to HTTP URI
MAC	String	**Required.** The WIFI Station MAC address String: AABBCCDDEEFF
seq	number	**Required.** the command sequence.
payload	Object	**Required.** locationId | String | Location ID of the system The remained payload part is the same as its corresponding HTTP command payload.

URI Outline

The REST interface is made up of a number of URIs that are used to control various aspects of the system. These URIs are listed below along with a short description and a link to the associated section of the document.

- / - Base URI, Unused.
  - /fixture – Used to interact with individual fixtures.
  - /group – Used to interact with groups.
  - /automation – Used to interact with automations (scenes)
  - /remote – Used to interact with BLE-based remote/wall control configurations.
  - /input – Used to interact with physical input configurations.
  - /fs – Used to interact with the file system of the device.
  - /ota – Used to perform updates of the processors within the device.
  - /network – Used to interact with the network connectivity capabilities of the device.
  - /device – Used to interact with the device configuration.
  - /integration – Used to interact with local integrations.
  - /debug – Provides real time debug information from the system.

Fixture

To interact with individual fixtures within the system, the URI /fixture should be used.

The fixture URI has the following action types. Each action type is explained in further detail in the associated section.

Action	Identifier	Overview
Create	0	Has no effect on fixtures.
Modify	1	Modifies an existing fixture.
Delete	2	Deletes an existing fixture.
Read	3	Read specifics about an existing fixture.
Control	4	Control an existing fixture.
List	5	List all existing fixtures.
Configure	6	Configure an existing fixture.
Search	7	Initiates a search for new fixtures.

Create

A fixture is created by inserting the fixture into the track and allowing it to be identified. There is nothing to be done to create a fixture.

Response

Content Type: application/json

Code	Description
400	Bad Request: Returned when the request was not properly formatted.

Modify

Modifying a fixture allows the customized parameters to be adjusted.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for modify it is 1.
addr	Number	**Required.** Address of the fixture to be modified.
name	String	**Required.** Desired name for the fixture. Maximum length of the name is 32 characters.

Example JSON to modify an existing fixture:

```
{
```

```
        "action": 1,
        "addr": x,
        "name": "example"
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of modify, action is 1.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to modifying a fixture:

```
{
        "action": 1,
        "result": "x"
}
```

Delete

Deleting a fixture removes the fixture from the system resulting in no further control or information about the fixture.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for delete it is 2.
addr	Number	**Required.** Address of the fixture to be

		deleted.

Example JSON to delete a fixture:

```
{
        “action”: 2,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of delete, action is 2.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to deleting a fixture:

```
{
        “action”: 2,
        “result”: “x”
}
```

Read

Read the specifics associated with a fixture.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the

		action to perform, for read it is 3.
addr	Number	**Optional.** Address of the fixture to read. May be a single address (Number) or an array of addresses. If omitted, or supplied as an empty array, all fixtures are returned in a fixture array along with a count field.

Example JSON to read a fixture:

```
{
        "action": 3,
        "addr": x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of read, action is 3.
addr	Number	**Optional**. In the case of a single-fixture read, the address of the fixture that was read. In the case of failure, this will not be present.
fixture	Array of Objects	**Optional**. Present when multiple fixtures are read (addr supplied as an array, empty array, or omitted). Each element is an object containing the addr, name, type, state, tune, and detail of one fixture.
count	Number	**Optional**. Present when all fixtures are returned; the number of fixtures included in the fixture array.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
name	String	**Optional.** In the case of success, the name of the fixture. In the case of failure, this will not be present.
type	Number	**Optional.** In the case of success, an

		enumeration corresponding to the type of the fixture. In the case of failure, this will not be present.
state	Object	**Optional.** In the case of success, an object containing a control structure associated with the type of the fixture that was read, see Appendix 1 – Fixture Types for more information. In the case of failure, this will not be present.
tune	Object	**Optional.** In the case of success, an object containing a configuration structure associated with the type of the fixture that was read, see Appendix 1 – Fixture Types for more information. In the case of failure, this will not be present.
detail	Object	**Optional.** In the case of success, an object containing a detail structure associated with the type of the fixture that was read, see Appendix 1 – Fixture Types for more information. In the case of failure, this will not be present.

Example JSON response to reading a fixture:

```
{
        “action”: 3,
        “result”: “x”,
        “name”: “example”,
        “type”: y,
        “state”: { <structure associated with the type of fixture> },
        “tune”: { <structure associated with the type of fixture> },
        “detail”: { <structure associated with the type of fixture> }
}
```

Control

Controlling a fixture sets the state of the fixture.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for control it is 4.
addr	Number	**Required.** Address of the fixture to control.
state	Object	**Required.** Object containing the state

		associated with the type of fixture being controlled, see Appendix 1 – Fixture Types for more information.

Example JSON to control a fixture:

```
{
        “action”: 4,
        “addr”: x,
        “state”: { <structure associated with the type of fixture> }
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of control, action is 4.
status	String	**Optional**. When present, a human-readable diagnostic message describing the result of the action (for example, a validation error). Not present in all responses.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
state	Object	**Optional.** In the case of success, an object containing a structure associated with the type of the fixture that was read, see Appendix 1 – Fixture Types for more information. In the case of failure, this will not be present.

Example JSON response to controlling a fixture:

```
{
        “action”: 4,
        “result”: “x”,
        “state”: { <structure associated with the type of fixture> }
}
```

List

Listing the fixtures returns the address of all the fixtures for the system.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for list it is 5.

Example JSON to list the fixtures:

```
{
        “action”: 5
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of list, action is 5.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
addr	Array of Numbers	**Optional.** In the case of success, a numerical list of the addresses of all fixtures. In the case of failure, this will not be present.

Example JSON response to listing the fixtures:

```
{
        “action”: 5,
        “result”: “x”,
        “addr”: [ fixture_addr1, fixture_addr2, … ]
```

```
}
```

Configure

Configuring a fixture sets the fine-tuning values for the fixture.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for configure it is 6.
addr	Number	**Required.** Address of the fixture to configure.
tune	Object	**Required.** Object containing the tuning values associated with the type of fixture being configured, see Appendix 1 – Fixture Types for more information.

Example JSON to control a fixture:

```
{
        “action”: 4,
        “addr”: x,
        “tune”: { <structure associated with the type of fixture> }
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of configure, action is 6.
status	String	**Optional**. When present, a human-readable diagnostic message describing the result of

		the action (for example, a validation error). Not present in all responses.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
tune	Object	**Optional.** In the case of success, an object containing a structure associated with the type of the fixture that was read, see Appendix 1 – Fixture Types for more information. In the case of failure, this will not be present.

Example JSON response to configuring a fixture:

```
{
        “action”: 6,
        “result”: “x”,
        “tune”: { <structure associated with the type of fixture> }
}
```

Search

Search initiates the process of finding new fixtures.

Syntax

HTTP Method: POST
URI: /fixture

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for search it is 7.

Example JSON to list the fixtures:

```
{
        “action”: 7
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred

	before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of search, action is 7.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to listing the fixtures:

```
{
        “action”: 7,
        “result”: “x”
}
```

Result Enumeration

The response to all remote actions that accompany a status of 200 contain a result enumeration. The possible values for that enumeration are found in Appendix 3: Error Codes

Group

To interact with groups within the device, the URI /group should be used.

The group URI has the following action types. Each action type is explained in further detail in the associated section.

Action	Identifier	Overview
Create	0	Creates a new group.
Modify	1	Modifies an existing group.
Delete	2	Deletes an existing group.
Read	3	Read specifics about an existing group.
Control	4	Control an existing group.
List	5	List all existing groups.
Create Multiple Groups	11	Removes all existing groups and then creates multiple groups in one command
Remove All Groups	20	Remove all existing groups

Create

Creating a group takes a list of fixtures and customized parameters, and assigns them an address in the device so that they can be control as a single entity.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for create it is 0.
name	String	**Required.** Desired name for the group. Maximum length of the name is 32 characters.
fixture	Array of Numbers	**Required.** Numerical list of the addresses to include in the group.
addr	Number	Optional. If supplied, creates a local group at this specific address instead of having one auto-assigned.
locationGroupVersion	Number	Optional. Version value for the group configuration; stored when supplied.

Example JSON to create a new group:

```
{
```

```
        "action": 0,
        "name": "example",
        "fixture": [ fixture_addr1, fixture_addr2, … ]
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of create, action is 0.
status	String	Optional. When present, a human-readable diagnostic message describing the result of the action (for example, a validation error). Not present in all responses.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
addr	Number	**Optional.** In the case of success, a number corresponding to the group's address will be returned. In the case of failure, this will not be present.

Example JSON response to creating a new group:

```
{
        "action": 0,
        "result": "x",
        "addr": y
}
```

Modify

Modifying a group allows the customized parameters and/or the list of fixtures that have been assigned to an address to be adjusted.

Syntax

HTTP Method: POST

URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for modify it is 1.
addr	Number	**Required.** Address of the group to be modified.
name	String	**Optional.** Desired name for the group. Maximum length of the name is 32 characters.
fixture	Array of Numbers	**Optional.** Numerical list of the addresses to include in the group.
locationGroupVersion	Number	Optional. Version value for the group configuration; stored when supplied.

Example JSON to modify an existing group:

```
{
        “action”: 1,
        “addr”: x,
        “name”: “example”,
        “fixture”: [ fixture_addr1, fixture_addr2, … ]
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of modify, action is 1.
status	String	Optional. When present, a human-readable diagnostic message describing the result of the action (for example, a validation error).

		Not present in all responses.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to modifying a group:

```
{
        “action”: 1,
        “result”: “x”,
        “addr”: y
}
```

Delete

Deleting a group removes the linking of the fixtures to the assigned address.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for delete it is 2.
addr	Number	**Required.** Address of the group to be deleted.

Example JSON to delete a group:

```
{
        “action”: 2,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of delete, action is 2.
status	String	**Optional**. When present, a human-readable diagnostic message describing the result of the action (for example, a validation error). Not present in all responses.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to deleting a group:

```
{
        “action”: 2,
        “result”: “x”
}
```

Read

Read the specifics associated with a group.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for delete it is 3.
addr	Number	**Optional.** Address of the group to read. May be a single address (Number) or an array of addresses. If omitted, all groups are returned in a group array. Address 255 is the built-in “All-Default” group, which returns all fixtures.

Example JSON to read a group:

```
{
        “action”: 3,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description

200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of read, action is 3.
addr	Number	**Optional**. For a single-group read, the address of the group that was read. Not present on failure.
group	Array of Objects	**Optional**. Present when multiple groups are read (addr supplied as an array, or omitted). Each element is an object containing the addr, name, and fixture list of one group.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
name	String	**Optional.** In the case of success, the name of the group. In the case of failure, this will not be present.
fixture	Array of Numbers	**Optional.** In the case of success, a numerical list of the addresses that are in the group. In the case of failure, this will not be present.

Example JSON response to reading a group:

```
{
        “action”: 3,
        “result”: “x”,
        “name”: “example”,
        “fixture”: [ fixture_addr1, fixture_addr2, … ]
}
```

Control

Controlling a group allows all members of a group to controlled simultaneously using a union of the features that the members can perform.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for control it is 4.
addr	Number	**Required.** Address of the group to control.
state	Object	**Required.** Object containing the union of actions for the members of the group. See Appendix 1 – Fixture Types for the available control fields.

Example JSON to control a group:

```
{
        “action”: 4,
        “addr”: x,
        “state”: { <structure associated with the union of actions the group members can perform> }
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of control, action is 4.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to controlling a group:

```
{
        “action”: 4,
        “result”: “x”
}
```

List

Listing the groups returns the address of all of the groups that have been created.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for list it is 5.

Example JSON to list the groups:

```
{
        “action”: 5
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of list, action is 5.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
addr	Array of Numbers	**Optional.** In the case of success, a numerical list of the addresses of all created groups. In the case of failure, this will not be present. The returned list always includes the built-in “All-Default” group at address 255.

Example JSON response to listing the groups:

```
{
        “action”: 5,
        “result”: “x”,
        “addr”: [ group_addr1, group_addr2, … ]
```

```
}
```

Create Multiple Groups

Creating multiple groups removes all existing groups and then creates each group in the supplied list in a single operation.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for create multiple groups it is 11.
groups	Array of Objects	**Required.** Array of group objects to create. Each object contains name and fixture (and optionally addr), as in the Create action.
locationGroupVersion	Number	**Optional.** Version value for the group configuration; stored when supplied.

Example JSON to create Multiple Groups:

```
{
“action”: 11,
“groups”: [ { “name”: “example”, “fixture”: [ … ] }, … ]
}
```

Response

Content Type: application/json

Code	Description
400	Bad Request: Returned when the request was not properly formatted.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of create multiple groups, action is 11.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response:

```
{
“action”: 11,
“result”: “x”
}
```

Remove All Groups

Removing all groups deletes every group that has been created on the device.

Syntax

HTTP Method: POST
URI: /group

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for remove all groups it is 20.

Example JSON to remove All Groups:

{
“action”: 20
}

Response

Content Type: application/json

Code	Description
400	Bad Request: Returned when the request was not properly formatted.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of remove all groups, action is 20.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response:

{
“action”: 20,
“result”: “x”
}

Result Enumeration

The response to all remote actions that accompany a status of 200 contain a result enumeration. The possible values for that enumeration are found in Appendix 3: Error Codes

Automation

To interact with automations within the device, the URI /automation should be used.
Note that automations are sometimes called scenes. The URI /scene was formerly used for this functionality.

Types
There are three types of Automations:

- **Standard** – these are triggered by a user action (using the mobile app for instance) or by a schedule. A standard automation sets groups and fixtures to a particular state.
- **Occupancy** – these are triggered by an occupancy sensor state change. These set groups or fixtures to a state depending on the state of the occupancy sensors: occupied, vacant or disabled (so these automations specify three states)
- **Daylight** – A daylight automation specifies one group that is to be kept within a range of measured illuminance values by changing the state of the group to be more or less bright.

Order of Operations for fixtures and groups
If a fixture is in a group and both the fixture and group are part of an automation, the order of operations is to run the group changes first and then the fixture changes. So, the individual fixture settings will override the group settings.

Allow for no-op occupancy states
For occupancy automations, there are 3 states that can be set: occupied, vacant, and disabled. If the fixture and group fields for a particular state (like occupied) are left blank, then that state will have no affect on any of the fixtures. This is to allow for cases such as "have the sensors turn the lights OFF for vacant, but do NOT turn the lights on for occupied. The user will turn the lights on manually if they are needed".

What happens when an automation is manually overridden?
After an automation sets the fixtures to particular states, the user could override these states, either with a manual operation (mobile app), or with another scheduled automation. The results are different for different automation types.

- **Standard** – A manual override will stick. A standard automation only runs once and then is done. Nothing will try and "re-set" the standard automation.
- **Occupancy** – If an occupancy automation is overridden, the changes will stay until the next change of state. At the next change of state the automation will override the manual changes.
- **Daylight** – TBD

The automation URI has the following action types. Each action type is explained in further detail in the associated section.

Action	Identifier	Overview
Create	0	Creates a new automation.
Modify	1	Modifies an existing automation.
Delete	2	Deletes an existing automation.
Read	3	Read specifics about an existing automation.
Control	4	Control an existing automation.
List	5	List all existing automations.
Dimming	6	Dim the fixtures which in the automation
Create Multiple Automations	10	Remove all existing automations and create multiple new ones

Create

Creating an automation takes the customized parameters, a list of fixtures, and the fixture states, and assigns them an address in the device so that the assigned state can all be entered simultaneously.

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

<table>
<tr><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>action</td><td>Number</td><td>Required. Identifier corresponding to the action to perform, for create it is 0.</td></tr>
<tr><td>name</td><td>String</td><td>Required. Desired name for the automation. Maximum length of the name is 32 characters.</td></tr>
<tr><td>type</td><td>Number</td><td>Optional. Type of Automation. Default is 1=Standard
An enumeration used to tell what type of Automation.
1 = standard
2 = occupancy
3 = daylight
All other values are reserved and should not be used</td></tr>
<tr><td>enabled</td><td>Boolean</td><td>Optional. Default is True. Set to False to disable this automation</td></tr>
<tr><td>locationAutomationVersion</td><td>Number</td><td>Optional. Version value for the automation configuration; stored when supplied.</td></tr>
<tr><td colspan="3">The fields below are required when the automation type is “standard”</td></tr>
<tr><td>fixture</td><td>Array of Objects</td><td>For standard automations, one fixture entry OR one group entry is Required. Array of objects containing the address and state the of the fixture.
<table>
<tr><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>addr</td><td>Number</td><td>Required. Address of the fixture to be added. See Appendix 1 – Fixture Types for the available state fields.</td></tr>
<tr><td>state</td><td>Object</td><td>Required. Object containing the state with the attributes that want to be adjusted. See</td></tr>
</table></td></tr>
</table>

<table>
<tr><td></td><td></td><td colspan="3"><table><tr><td></td><td></td><td>Appendix 1 – Fixture Types for more information. Any attributes that are not desired to be changed by the automation should not be specified.</td></tr></table></td></tr>
<tr><td>group</td><td>Array of Objects</td><td colspan="3">For standard automations, one fixture entry OR one group entry is Required. Array of objects containing the address and state the of the fixture.<table><tr><th>Name</th><th>Type</th><th>Description</th></tr><tr><td>Addr</td><td>Number</td><td>Required. Address of the fixture to be added.</td></tr><tr><td>State</td><td>Object</td><td>Required. Object containing the state with the attributes that want to be adjusted. See Appendix 1 – Fixture Types for more information. Any attributes that are not desired to be changed by the automation should not be specified. Changed by the automation should not be specified.</td></tr></table></td></tr>
<tr><td colspan="4">The fields below are required when the automation type is “occupancy”</td></tr>
<tr><td>occSensors</td><td>Array of Numbers</td><td colspan="3">Required for occupancy automations. Not used for standard or daylight automations. For occupancy automations this is a list of occupancy sensors. Array of addresses representing occupancy sensor addresses. These occupancy sensors are used to trigger the automation, based on the sensor reported values.

If the automation state is unoccupied, and any sensor switches to occupied state, the automation state is changed to occupied and the occupied state is executed (applied to the fixtures and groups).

If the automation state is occupied, and ALL</td></tr>
</table>

		sensors switch to unoccupied state, the automation state is changed to unoccupied and the unoccupied state is executed (applied to the fixtures and groups). When the automation is changed to disabled, the disabled state will be run. When the automation is changed from disabled to enabled, the automation state is set to unoccupied and the unoccupied scene is run.
fixture	Array of Objects	**Not used for standard.** Contains 3 objects: {“occupied”:[]}, {“vacant”:[]}, {“disabled”:[]} each of which contains a list of fixture objects.
group	Array of Objects	**Not used for standard.** Contains 3 objects: {“occupied”:[]}, {“vacant”:[]}, {“disabled”:[]} each of which contains a list of group objects.
occupied	Array of Objects	**Not used for standard automations.** List of fixtures or groups and their states when in the occupied state.
vacant	Array of Objects	**Not used for standard automations.** List of fixtures or groups and their states when in the vacant state.
disabled	Array of Objects	**Not used for standard automations.** List of fixtures or groups and their states when in the disabled state.
fixture	Array of Objects	**Not used for standard.** Contains 3 objects: {“occupied”:[]}, {“vacant”:[]}, {“disabled”:[]} each of which contains a list of fixture objects.
The fields below are required when the automation type is “daylight”		
lightSensor	Array	**Required for daylight automations. Not used for standard or occupancy automations.** This tells the address of the light sensor to use to sense the light level.
daylightGroup	Number	**Required for daylight automations. Not used for standard or occupancy automations.** This tells which group will have its light level adjusted to try and reach the desired light level
minTargetLevel	Number	**Required for daylight automations. Not used for standard or occupancy automations.**
maxTargetLevel	Number	**Required for daylight automations. Not used for standard or occupancy automations.**

Example JSON to create a new **standard** automation using fixtures:

```
{
```

```
        “action”: 0,
        “name”: “front lights on”,
        “type”: 1,
        “fixture”:
        [
                {
                        “addr”: 1,
                        “state”: {“status”: true, “level”:5000}
                },
                {
                        “addr”: 2,
                        “state”: {“status”: true, “level”:3000}
                },
                …
        ]
}
```

Example JSON to create a new **standard** automation using groups:

```
{
        “action”: 0,
        “name”: “groups on”,
        “type”: 1
        “group”:
        [
                {
                        “addr”: 1,
                        “state”: {“status”: true, “level”:5000}
                },
                {
                        “addr”: 2,
                        “state”: {“status”: true, “level”:3000}
                },
                …
        ]
}
```

Example JSON to create a new **standard** automation using fixtures AND groups:

```
{
        “action”: 0,
        “name”: “front and back on”,
        “type”: 1,
        “fixture”:
        [
                {
                        “addr”: 1,
                        “state”: {“status”: true, “level”:5000}
                },
                {
                        “addr”:2,
```

```
                        “state”: {“status”: true, “level”:3000}
                }
        ],
        “group”:
        [
                {
                        “addr”: 1,
                        “state”: {“status”: true, “level”:10000}
                },
                {
                        “addr”: 2,
                        “state”: {“status”: true, “level”:10000}
                }
        ]
}
```

Example JSON to create a new **occupancy** automation, on at 75% when occupied, on at 10% when unoccupied, off when disabled.

```
{
        "action": 0,
        "name": "with occupancy",
        "type": 2,
        "occSensors": [100],
        "fixture": {
           "occupied": [{"addr": 1, “state": {"status": true,  "level": 7500}}],
           "vacant": [{"addr": 1, "state": {"status": true, "level": 1000}}],
           "disabled": [{"addr": 1, "state": {"status": false}}]
        },
        "group": {
           "occupied": [{"addr": 1, "state": {"status": true,  "level": 7500}}],
           "vacant": [{"addr": 1, "state": {"status": true, "level": 1000}}],
           "disabled": [{"addr": 1, "state": {"status": false}}]
        }
}
```

Example JSON to create a new **daylight** automation, with target level between 1000 and 1200 for group with address 4

```
{
        “action”: 0,
        “name”: “lunch room”,
        “type”: 3,
“lightSensor”: [3],
        “daylightGroup”: 4,
“minTargetLevel”: 1000,
“maxTargetLevel”: 1200
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
Action	Number	**Required.** Echo of the action performed. In the case of create, action is 0.
Result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
Addr	Number	**Optional.** In the case of success, a number corresponding to the automation's address will be returned. In the case of failure, this will not be present.

Example JSON response to creating a new automation:

```
{
        "action": 0,
        "result": "x",
        "addr": x
}
```

Modify

Modifying an automation allows the user to change values of the automation. A modify command requires only the action and addr fields. The user can modify a single field by including it in the modify command, or can modify multiple fields by including multiple fields in the modify command. So in order to modify an automation the user does not need to include all fields for that automation type, just the ones that the user wants to change.

Syntax

HTTP Method: POST
URI: /automation

Parameters

The parameters are the same as for the Create function. The only required parameters are the action and addr fields and any fields that are to be modified.

Examples of a modify are the same as create but with only action and addr as required parameters.

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of modify, action is 1.

Example JSON response to modifying an automation:

```
{
        “action”: 1,
        “result”: “x”,
        “addr”: y
}
```

Delete

Deleting an automation removes the linking of the fixtures to the assigned address.

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for delete it is 2.
addr	Number	**Required.** Address of the automation to be deleted.

Example JSON to delete a group:

```
{
        “action”: 2,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of delete, action is 2.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to deleting an automation:

```
{
        “action”: 2,
        “result”: “x”
}
```

Read

Read the specifics associated with an automation.

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for delete it is 3.
addr	Number	**Required.** Address of the group to read.

Example JSON to read an automation:

```
{
        “action”: 3,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:
The format contains the same parameters as the Create function. With action and result being required.

Example JSON response to reading a **standard** automation:

```
{
        “action”: 3,
        “result”: x,
        “name”: “front lights on”,
        “type”: 1,
        “fixture”:
        [
                {
                        “addr”: 1,
                        “state”: {“status”: true, “level”:5000}
                },
                {
                        “addr”: 2
                        “state”: {“status”: true, “level”:3000}
                },
                …
        ]
}
```

Example JSON response to reading an **occupancy** automation, on at 75% when occupied, on at 10% when unoccupied, off when disabled.

```
{
        "action": 3,
        “result”:x,
        "name": "with occupancy",
```

```
            "type": 2,
            "occSensors": [100],
            "fixture": {
               "occupied": [{"addr": 1, “state": {"status": true,  "level": 7500}}],
               "vacant": [{"addr": 1, "state": {"status": true, "level": 1000}}],
               "disabled": [{"addr": 1, "state": {"status": false}}]
            },
            "group": {
               "occupied": [{"addr": 1, "state": {"status": true,  "level": 7500}}],
               "vacant": [{"addr": 1, "state": {"status": true, "level": 1000}}],
               "disabled": [{"addr": 1, "state": {"status": false}}]
            }
}
```

Example JSON response to reading a **daylight** automation, with target level between 1000 and 1200 for group with address 4

```
{
            “action”: 3,
            “result”:x
            “name”: “lunch room”,
            “type”: 3,
“lightSensor”: [3],
            “daylightGroup”: 4,
“minTargetLevel”: 1000,
“maxTargetLevel”: 1200
}
```

Control

Controlling an automation sets all members of the automation to their specified states in unison.

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for control it is 4.
addr	Number	**Required.** Address of the automation to control.

Example JSON to control a group:

```
{
        “action”: 4,
        “addr”: x
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of control, action is 4.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to controlling an automation:

```
{
        “action”: 4,
        “result”: “x”
}
```

List

Listing the automations returns the address of all the automations that have been created.

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for list it is 5.

Example JSON to list the automations:

```
{
        “action”: 5
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of list, action is 5.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.
addr	Array of Numbers	**Optional.** In the case of success, a numerical list of the addresses of all created automations. In the case of failure, this will not be present.

Example JSON response to listing the automations:

```
{
        “action”: 5,
        “result”: “x”,
        “addr”: [ automation_addr1, automation_addr2, … ]
}
```

Dimming

Dim the fixtures or groups in the automation. This let the fixtures level up/down with 5 percent. Only used in invisiLED

Syntax

HTTP Method: POST
URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the

		action to perform, for dimming it is 6.
addr	Number	**Required.** Address of the automation to control.
dimming	Number	**Required.** Dimming Value. it default 5 for dimming up, -5 for dimming down.

Example JSON to list the automations:

```
{
        “action”: 6,
        “addr”:1,
        “dimming”:5
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of list, action is 6.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response to listing the automations:

```
{
        “action”: 6,
        “result”: “x”
}
```

Create Multiple Automations

Creating multiple automations first removes all existing automations (except the built-in all-off automation) and then creates each automation in the supplied list in a single operation.

Syntax

HTTP Method: POST

URI: /automation

Parameters

Content Type: application/json

Name	Type	Description
action	Number	**Required.** Identifier corresponding to the action to perform, for create multiple automations it is 10.
automation	Array of Objects	**Required.** Array of automation objects to create. Each object is a full automation as in the Create action.
locationAutomationVersion	Number	**Optional.** Version value for the automation configuration; stored when supplied.

Example JSON to create multiple automations:

{
“action”: 10,
“automation”: [{ … }, { … }]
}

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted.
408	Request Timeout: Returned when a socket timeout occurred before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Format of the JSON structure returned with a response status of 200:

Name	Type	Description
action	Number	**Required.** Echo of the action performed. In the case of create multiple automations, action is 10.
result	String	**Required.** String representation of an enumeration corresponding to the success or failure of the action.

Example JSON response:

{

“action”: 10,
“result”: “x”
}

Result Enumeration

The response to all remote actions that accompany a status of 200 contain a result enumeration. The possible values for that enumeration are found in Appendix 3: Error Codes

Device

To interact with the configuration of the device the /device URI is used.

When interacting with /device the returned data depends directly on the input parameters that are provided.

Syntax

HTTP Method: POST
URI: /device

Parameters

Content Type: application/json

Name	Type	Description
owner	String	**Read-only**. The email address of the device owner. This value is read-only through this interface; if it is sent, the provided value is ignored.
deviceName	String	**Optional.** If sent the device name will be changed to match the provided value and device information will be returned. The value is a string containing a device name that has a length of between 1 and 64 characters, inclusive. If a zero length value is specified, a result of "-3" shall be returned. If a value larger than 64 is specified, a result of "-18" shall be returned.
staMac	String	**Optional.** If sent the device information will be returned, but the provided value will be ignored as it cannot be changed.
apMac	String	**Read-only**. The MAC address of the device's Wi-Fi access point interface. If this parameter is included in a request, the request is rejected and a read-only-parameter error is returned.
bleMac	String	**Read-only**. The MAC address of the device (Bluetooth Low Energy). If this parameter is included in a request, the request is rejected and a read-only-parameter error is returned.
dateCode	String	**Optional.** If sent the stored value for the manufacturing date will be changed to match the provided value and device information will be return. The date code should have a format of yyyy-mm-dd.

		Note: This is a special value and has the additional requirement of needing to be preceded by the manufacturing parameter.
iotmVer	String	**Read-only**. The version of the IoT module firmware. If this parameter is included in a request, the request is rejected and a read-only-parameter error is returned.
restVer	String	**Read-only**. The version of the REST interface. If this parameter is included in a request, the request is rejected and a read-only-parameter error is returned.
scmVer	String	**Read-only**. The version of the SCM firmware. If this parameter is included in a request, the request is rejected and a read-only-parameter error is returned.
time	String	**Optional.** If sent the device with value string is empty. The device will return the time string by the format: yyyy-mm-dd hh:mm:ss If the value string is available. Will set the device time. the time string format is: yyyy-mm-dd hh:mm:ss
timeZone	String	**Optional.** If sent the time zone will be changed to match the provided value and device information will be return. The time zone is sent in the following format: tzn[+|-]hh[:mm[:ss]][dzn]. Further information can be found here.
query	Any	**Optional.** If sent the device information will be returned. The input has no effect on the system and the value will not be used.
factoryReset	Any	**Optional.** If sent the device will perform a factory reset and an acknowledgment will be returned.
hardFactoryReset	Any	**Optional.** If sent the device will perform a hard factory reset and there will be no return as the device will immediately reset. A hard factory reset differs from a regular reset in that the device does not wait for proper decommissioning of AWS, it simply resets itself to factory defaults.
reboot	Any	**Optional.** If sent the device will immediately reboot and there will be

		no return.
manufacturing	Boolean	**Optional.** Allows for manufacturing related information to be written if this value is true and those fields proceed this field in the JSON structure. An acknowledgment will be sent in response.
systemMode	Number	**Optional.** 1:PLC 2: TRIAC
discoverDev	Bool	**Optional.** True: Start Discover device False: Stop discover device
action	String	**Optional**. The action to perform. The only supported value is “read”, which returns the device information (equivalent to query).
bootCount	String	**Optional**. The only supported value is “reset”, which resets the stored boot counter. Device information is returned.
latitude	Number	**Optional**. Sets the device’s latitude in tenths of a degree. Values outside the supported range return an error.
longitude	Number	**Optional**. Sets the device’s longitude in tenths of a degree. Values outside the supported range return an error.
locationId	String	**Optional**. Sets the device’s location identifier. A value of “CLEAR” clears the stored location ID.
networkState	Number	**Optional**. Sets the network LED state to the provided numeric value.
plcVer	String	**Optional**. Returns PLC version information: PLC version, date, role/protocol profile, extended version, chip type, and SCM version.
plcReboot	String	**Optional**. Reboots the PLC chip. Returns an action of “plcReboot” and a plcStatus value indicating the result.
SddpNotifyID	String	**Optional**. Triggers a Control4 SDDP identify notification. Device information is returned.
fetchDebug	String	**Optional**. Initiates a full device shadow update to the cloud and returns a result string confirming the update was initiated.
testModelName	String	**Optional**. Debug/manufacturing only. Sets a test model name.
testRevModelName	String	**Optional**. Debug/manufacturing only. Sets a test revision model name.
plcTest	String	**Optional**. Debug only. Triggers a PLC test; the result is returned under

		plcTest.
dbgTimeZone	String	**Optional**. Debug only. Sets the time zone for debugging purposes.
systemSpecificParams	**Objects**	**Optional.** Specific parameters of the system

The chart below show the detailed information about the systemSpecificParams item

naturalLightingInfo	Objects	calcPeriodSec	Number	The period of calculate the natural information
		minCCTDelta	Number	Minmin delta of the color temperature
		minLevelDelta	Number	Minmin delta of the level
		minSendFreq	Number	Minmin frequency of send natural information

Note: One or all of the parameters can be sent in the same message.

Note: The return JSON structure will correspond to the last valid field of the input JSON structure.

Example JSON showing inputs and their associated data format:

```
{
        “owner”: “example@waclighting.com”,
        “deviceName”: “example”,
        “staMac”: “00:00:00:00:00:00”,
        “apMac”: “00:00:00:00:00:00”,
        “bleMac”: “00:00:00:00:00:00”,
        “dateCode”: “2020-01-01”,
        “iotmVer”: “00.00.0000”,
        “restVer”: “0.00”,
        “scmVer”: “00.00.0000”,
        “time”: “Mon Mar  2 00:00:00 2020”,
        “timeZone”: “UTC+6:00”,
        “query”: 1,
        “factoryReset”: 1,
        “hardFactoryReset”: 1,
        “reboot”: 1,
        “manufacturing”: true
}
```

Response

Content Type: application/json

Code	Description
200	Success: Returned when the command was recognized and attempted. Payload will contain a JSON structure with further information.
400	Bad Request: Returned when the request was not properly formatted or none of the fields were valid.
408	Request Timeout: Returned when a socket timeout occurred

	before all data could be received.
500	Internal Server Error: Returned when there was an error within the device.

Device Information

In addition to the fields listed below, the fields from the Request section that have values will also be returned in the response. Format of the device information JSON structure returned with a response status of 200:

Name	Type	Description
owner	String	**Required.** The email address of the own that has been assigned to the device.
deviceName	String	**Required.** The name that has been assigned to the device.
staMac	String	**Required.** MAC address for the device when WiFi is operating in station mode. Will have the format: XX:XX:XX:XX:XX:XX.
apMac	String	**Required.** MAC address for the device when WiFi is operating in access point mode. Will have the format: XX:XX:XX:XX:XX:XX.
bleMac	String	**Required.** BLE MAC address for the device. Will have the format: XX:XX:XX:XX:XX:XX.
dateCode	String	**Required.** Date that the device was manufactured. Will have the format: yyyy-mm-dd.
iotmVer	String	**Required.** Firmware version of the IOTM processor in the system. Will have the format: xx.xx.xxxx.
restVer	String	**Required.** Version of the REST interface that the system is using. Will have the format: x.xx.
scmVer	String	**Required.** Firmware version of the SCM processor in the system. Will have the format: xx.xx.xxxx.
time	String	**Required.** Current time as determined using the set time zone and SNTP. Will have the format of: yyyy-mm-dd hh:mm:ss
timeZone	String	**Required.** Current time zone that the device is configured to use.
accessoryType	Number	**Required** current peripheral module socked into the brain. 0:none 1:0-10V ADC module 2: dmx module
bootCount	Number	The number of times the device has booted.
uptimeSeconds	Number	Device uptime in seconds since the last boot.
locationId	String	The device's location identifier.
systemType	Number	The system type of the device. Possibilities: strut colorscaping gen3fan
builtFor	String	Identifies if the firmware was built for real hardware for development hardware. Possibilities: strutHw devKit

tzoffset	Number	Current time-zone offset from UTC, in seconds.
latitude	Number	The device's configured latitude in tenths of a degree
longitude	Number	The device's configured longitude in tenths of a degree
networkState	Number	The current network LED state.
features	Array of String	Feature flags supported by the device. Values: localGrps, localAutm, wsMcast, cloudLocationTopics, scheduleOverrides; on non-colorscaping systems also dimToWarm and naturalLighting.
systemSpecificParams	Object	System-type-specific parameters. These change depending on the system type. Members: accessoryType (Number), discoverStatus (Number), panelVer (String), systemInitCompleted (Boolean), systemMode (Number), systemModeChangeable (Boolean), plcInfo (Object) and naturalLightingInfo (Object). plcInfo members: plcVer, plcDateInfo, plcRPP, plcExtVer, plcChipType. naturalLightingInfo members: calcPeriodSec, minSendFreq, minCCTDelta, minLevelDelta.
nwkState	Object	Network status. (see the /network documentation for more details) Members: provisioned, commissioned, ssid, ipAddr, netmask, connectMethod, ethIpAddr, staIpAddr, bssid, rssi, auth,

<table>
<tr><td></td><td></td><td>channel,
awsState,
awsCloudConnection,
awsConnected,
certificateID,
currFreeHeap,
lg_free_block.</td></tr>
<tr><td>multicastState</td><td>Number</td><td>Current wall-station multicast state.</td></tr>
<tr><td>multicastRx</td><td>Number</td><td>Count of multicast messages received.</td></tr>
<tr><td>lastMcastPayload</td><td>String</td><td>The most recent multicast payload received.</td></tr>
<tr><td>cdebug</td><td>Number</td><td>Debug only. Internal debug value.</td></tr>
<tr><td>esp-idf-version</td><td>String</td><td>Debug only. The ESP-IDF framework version used to build the firmware.</td></tr>
<tr><td>awsRemoteCmdTxFail</td><td>Number</td><td>Debug. Count of AWS remote commands sent that failed.</td></tr>
<tr><td>awsRemoteReceived</td><td>Number</td><td>Debug. Count of AWS remote commands received.</td></tr>
<tr><td>awsRemoteCmdSent</td><td>Number</td><td>Debug. Count of AWS remote commands sent.</td></tr>
<tr><td>RGBPreset</td><td>Array of Objects</td><td>Optional. WAC_CS Only.
<table>
<tr><th>Name</th><th>Type</th><th>Description</th></tr>
<tr><td>index</td><td>int</td><td>Required. Index of the preset</td></tr>
<tr><td>type</td><td>int</td><td>Required. The type of the preset
1: RGB model
2: CCT model
Others: not used</td></tr>
<tr><td>red</td><td>int</td><td>Optional. If the type is RGB model only. The R value of the RGB 0-255</td></tr>
<tr><td>green</td><td>int</td><td>Optional. If the type is RGB model only.
The G value of the RGB 0-255</td></tr>
<tr><td>blue</td><td>Int</td><td>Optional. If the type is RGB model only. The B value of the RGB 0-255</td></tr>
<tr><td>level</td><td>Int</td><td>Optional. If the type is CCT model only. Level of the fixture
0-10000</td></tr>
<tr><td>mixColorTemp</td><td>Int</td><td>Optional. If the type is CCT model only.
Mix color temperature of the fixture 0-6500</td></tr>
</table></td></tr>
</table>

Example JSON response for the device information:


```
{
  "apMac" : "80:7D:3A:0F:F4:76",
  "bleMac" : "80:7D:3A:0F:F4:77",
  "bootCount" : 385,
  "builtFor" : "strutHw",
  "cdebug" : false,
  "dateCode" : "2020-03-19",
  "deviceName" : "austinLab04",
  "esp-idf-version" : "4.4.4",
  "features" : [
    "localGrps",
    "localAutm",
    "wsMcast",
    "cloudLocationTopics",
    "scheduleOverrides",
    "dimToWarm",
    "naturalLighting"
  ],
  "iotmVer" : "01.04.0192",
  "lastMcastPayload" : "",
  "latitude" : 3030,
  "locationId" : "478",
  "longitude" : -9775,
  "multicastRx" : 0,
  "multicastState" : 0,
  "networkState" : 3,
  "nwkState" : {
    "auth" : "WPA2_PSK",
    "awsCloudConnection" : "init",
    "awsConnected" : 1,
    "awsState" : "up",
    "bssid" : "A0:63:91:90:F3:5F",
    "certificateID" : "8ae677ffa4cc60053bc5a809ce28ee5b8944c7c7ccb9fecb2c94f81e8541d89f",
    "channel" : 4,
    "commissioned" : true,
    "connectMethod" : "wifi",
    "currFreeHeap" : 4041651,
    "ethIpAddr" : "0.0.0.0",
    "ipAddr" : "10.0.0.8",
    "lg_free_block" : 3932160,
    "netmask" : "255.255.255.0",
    "provisioned" : true,
    "rssi" : "-37",
    "ssid" : "CEL",
    "staIpAddr" : "10.0.0.8"
  },
  "owner" : "matt@waclighting.com",
  "restVer" : "2.00",
```

```
  "result" : "0",
  "scmVer" : "03.43",
  "staMac" : "80:7D:3A:0F:F4:75",
  "systemSpecificParams" : {
    "accessoryType" : 0,
    "discoverStatus" : 1,
    "naturalLightingInfo" : {
      "calcPeriodSec" : 5,
      "minCCTDelta" : 100,
      "minLevelDelta" : 100,
      "minSendFreq" : 5
    }
  },
  "systemType" : "strut",
  "time" : "Thu Jun 11 01:39:15 2026",
  "timeZone" : "none",
  "tzoffset" : "none",
  "uptimeSeconds" : 53
  "RGBPreset" : [
    {
      "blue" : 0,
      "green" : 0,
      "index" : 1,
      "red" : 255,
      "type" : 2
    },
    {
      "index" : 2,
      "level" : 10000,
      "mixColorTemp" : 3000,
      "type" : 1
    }
  ]
}
```

Appendix1 – Fixture Types

Fixtures are assigned types based on their capabilities. Each type is associated with a unique control and configuration structure which allows the all fixture's capabilities to be controlled and configured.

The fixture types are assigned IDs as follows. Each fixture type is explained in further detail in the associated section.

Fixture Type	Identifier
Single Color Dimmable Light	0
Tunable White Dimmable Light	1
RGBW Dimmable Light	2
Motorized Trackhead – Single Color Dimmable Light	3
ELV Single Color Dimmable Light	6
Wall Station	11
24V Controller	12
Fan	13
Low power decorative light	14
High power decorative light	15

Single Color Dimmable Light

The single color dimmable light has a set color temperature and can have the state and brightness controlled.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
online	Boolean	Read-only. True if the fixture is currently online and reachable, false otherwise.
mode	Number	Read-only. Enumeration of the fixture's current control mode.
colormode	String	Read-only. Current color mode of the fixture ("DIM", "CCT", or "RGB").
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.
findme	Boolean	On/Off findme function True Start findme , False Stop findme

requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        “status”: true,
        “level”: 7501,
        “findme”:true
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** – **Description** 0 – Linear dimming curve. 1 – Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.

<table>
<tr><td>factory</td><td>Number</td><td>Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at.<table><tr><th>Value</th><th>Description</th></tr><tr><td>1</td><td>Dongguan</td></tr><tr><td>2</td><td>Vietnam</td></tr><tr><td>3</td><td>Mexico</td></tr><tr><td>4</td><td>New York</td></tr><tr><td>5</td><td>Georgia</td></tr><tr><td>6</td><td>Los Angels</td></tr></table></td></tr>
<tr><td>model</td><td>String</td><td>Model number of the fixture.</td></tr>
<tr><td>ledDriver</td><td>String</td><td>Model number of the LED driver.</td></tr>
<tr><td>currentOutput</td><td>Number</td><td>Rated current output level of the fixture in milliamps.</td></tr>
<tr><td>currentLevel</td><td>Number</td><td>Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.</td></tr>
<tr><td>pcbVer</td><td>String</td><td>Version of the PCB used for the fixture.</td></tr>
<tr><td>fwVer</td><td>String</td><td>Version of the firmware running on the fixture.</td></tr>
<tr><td>busVer</td><td>String</td><td>Version of the bus protocol running on the fixture.</td></tr>
<tr><td>devAbility</td><td>Array</td><td>Optional:
Specific functionality the fixture support
bit0:CCT
bit1:RGB
bit2:Dim to warm
bit3:Natural Dynamic
bit4: device Dynamic
bit5: sync dynamic</td></tr>
</table>

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”
}
```

Low power decorative light

The same with the tunable white dimmable light.

Hight power decorative light

The same with the tunable white dimmable light.

Tunable White Dimmable Light

The tunable white dimmable light allows the color temperature to be adjusted along with control of the state and brightness.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
online	Boolean	Read-only. True if the fixture is currently online and reachable, false otherwise.
mode	Number	Read-only. Enumeration of the fixture's current control mode.
colormode	String	Read-only. Current color mode of the fixture ("DIM", "CCT", or "RGB").
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.
colorTempLevel	Number	Value of the color temperature steps value between 1-7
mixColorTemp	Number	The value of Mix temperature It should not appear the same time.
DTWLevel	Number	Dim to warm level. Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000

		corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501. The DCM should adjust the color temperature to meet the Dim To Warm curve
findme	Boolean	On/Off findme function True Start findme , False Stop findme
requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        "status": true,
        "level": 7501,
          "findme":true,
         "colorTempLevel":100,
         "mixColorTemp":100
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** / **Description** 0 / Linear dimming curve. 1 / Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.
dimToWarm	Boolean	Optional: True: enable the functionality of dim to warm

		False: disable the functionality of dim to warm
dimMode	Number	Optional: 0: normal mode. CCT mode 1: fixed color temperature mode

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 Dongguan 2 Vietnam 3 Mexico 4 New York 5 Georgia 6 Los Angeles
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
schemVer	String	Version of the schematic used for the fixture.
pcbVer	String	Version of the PCB used for the fixture.
fwVer	String	Version of the firmware running on the fixture.
busVer	String	Version of the bus protocol running on the fixture.
colorTempStepsTable	Array of Objects	colorStepsIndex: Index of the color steps1-7 mixColorTemp: The Mix color temperature of the steps.0-65535

minColorTemp	Number	Optional: Min color temperature of the fixture
maxColorTemp	Number	Optional:max color temperature of the fixture

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”,
“colorTempStepsTable”:
[
        {
        “colorStepsIndex”:1,
        “mixColorTemp”:1000
        }

        {
        “colorStepsIndex”:2,
        “mixColorTemp”:3000
        }

        ]
        “minColorTemp”:1000,
        “maxColorTemp”:6500

}
```

RGBW Dimmable Light

The RGBW dimmable light allows the output color of the light to be set. The color temperature can also be set, but will only be considered when the color is set to white (255, 255, 255 in RGB). This light can also have the brightness and state controlled.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
online	Boolean	Read-only. True if the fixture is currently online and reachable,

		false otherwise.
mode	Number	Read-only. Enumeration of the fixture's current control mode.
colormode	String	Read-only. Current color mode of the fixture ("DIM", "CCT", or "RGB").
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.
red	Number	Red portion of the color definition using RGB. Value is between 0 and 255.
green	Number	Green portion of the color definition using RGB. Value is between 0 and 255.
blue	Number	Blue portion of the color definition using RGB. Value is between 0 and 255.
mixColorTemp	Number	Value of the color temperature in degrees Kelvin. Only valid when the color is white or (255, 255, 255) in RGB.
DTWLevel	Number	Dim to warm level. Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501. The DCM should adjust the color temperature to meet the Dim To Warm curve
findme	Boolean	On/Off findme function True Start findme , False Stop findme
mode	number	1:Tunable white 2:RGB 3:HSV 4:Dim to warm
hue	Number	Hue portion of the color definition using HSV(0-10000)
saturation	Number	Saturation portion of the color

		definition using HSV(0-10000)
DHModel	Number	Optional:The model of the double head fixture. 1: down light only 2: up light only 3: both(down and up)
marqueeType	Number	Optional: The type of the Marquee mode 0: disable 1: Marquee type 1 2: Marquee type 2
requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        “status”: true,
        “level”: 7501,
        “red”: 50,
        “green”: 50,
        “blue”: 255,
        “mixColorTemp”: 5000,
         “hue”:1000,
          “saturation”:5000,
        “findme”:true
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** / **Description** 0 – Linear dimming curve. 1 – Logarithmic dimming curve.

onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.
dimToWarm	Boolean	Optional: True: enable the functionality of dim to warm False: disable the functionality of dim to warm

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 Dongguan 2 Vietnam 3 Mexico 4 New York 5 Georgia 6 Los Angeles
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
schemVer	String	Version of the schematic used for the fixture.
pcbVer	String	Version of the PCB used for the fixture.
fwVer	String	Version of the firmware running on the fixture.

busVer	String	Version of the bus protocol running on the fixture.
devAbility	Array	Optional: Specific functionality the fixture support bit0:CCT bit1:RGB bit2:Dim to warm bit3:Natural Dynamic bit4: device Dynamic bit5: sync dynamic

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”
}
```

Motorized Trackhead – Single Color Dimmable Light

The motorized trackhead with single color dimmable light allows the trackhead to remotely controlled to point in the desired direction. The trackhead can adjust in two dimensions which allow the head to be tilted up and down and spun in a circle. The light is at a set color temperature and can have the state and brightness controlled.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
online	Boolean	Read-only. True if the fixture is currently online and reachable, false otherwise.
mode	Number	Read-only. Enumeration of the fixture’s current control mode.
colormode	String	Read-only. Current color mode of the fixture (“DIM”, “CCT”, or “RGB”).
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where

<table>
<tr><td></td><td></td><td>10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.</td></tr>
<tr><td>zoom</td><td>Number</td><td>Update. – 14 August 2025:
This is a value between 0 and 255
0== Narrow
255 == Wide</td></tr>
<tr><td>axis</td><td>Number</td><td>Enumeration representing the direction in which the track head should move. The enumeration is defined as follows where the value for up/down can be OR’ed with the value for right/left. However, right and left can’t sent together, and up and down can’t either.
<table>
<tr><th>Value</th><th>Description</th></tr>
<tr><td>16</td><td>Move the head right.</td></tr>
<tr><td>32</td><td>Move the head left.</td></tr>
<tr><td>64</td><td>Move the head down.</td></tr>
<tr><td>128</td><td>Move the head up.</td></tr>
</table>
Note: This value can’t be sent at the same time as tilt and/or pan values. If it is the values of tilt and/or pan will be ignored.</td></tr>
<tr><td>timeLr</td><td>Number</td><td>Value in ms that the head should move in the right/left direction.
Note: This value will only be considered if axis is also sent.</td></tr>
<tr><td>timeUd</td><td>Number</td><td>Value in ms that the head should move in the up/down direction.
Note: This value will only be considered if axis is also sent.</td></tr>
<tr><td>tilt</td><td>Number</td><td>Value of the tilt of the head between 0 and 180 degrees, where 0 is at the ceiling and 180 is at the floor.
Note: This value can’t be sent at the same time as axis, timeLr, and/or timeUd values. If it is it will be ignored in favor of axis, timeLr, and/or timeUd.</td></tr>
<tr><td>pan</td><td>Number</td><td>Value of the rotational position of the head between 0 and 359 degrees.
Note: This value can’t be sent at the same time as axis, timeLr, and/or timeUd values. If it is it will be ignored in favor of axis, timeLr, and/or timeUd.</td></tr>
<tr><td>findme</td><td>Boolean</td><td>On/Off findme function True Start findme , False Stop findme</td></tr>
<tr><td>preset</td><td>Number</td><td>Index of the preset position. Let the motor move to the present position. This value can’t be sent at the same time with</td></tr>
</table>

		pan,tile,timeud,timelr, axis, zoom.
UDEndPointFlag	Number	The motor arrived the Up down end point or not 1:arrived the end point. 0 didn't
LREndPointFlag	Number	The motor arrived the left right end point or not 1:arrived the end point. 0 didn't
motorRun	Boolen	Let the motor run or stop. True: make motor run . False: make motor stop.
requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        "status": true,
        "level": 7501,
        "zoom": 7501,
        "axis": 48,
        "timeLr": 100,
        "timeUd": 100,
        "tilt": 45,
        "pan": 270,
        "findme":true
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** / **Description** 0 / Linear dimming curve. 1 / Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.
presetInfo	Array of	position / Number / Index of the default position

	Objects	<table><tr><td>level</td><td>Number</td><td>Value of the light brightness</td></tr><tr><td>tilt</td><td>Number</td><td>Value of the tilt of the head between 0 and 180 degrees, where 0 is at the ceiling and 180 is at the floor.</td></tr><tr><td>pan</td><td>Number</td><td>Value of the rotational position of the head between 0 and 359 degrees.</td></tr><tr><td>zoom</td><td>Number</td><td>Value allowing the size of the light spot to be adjusted between 0% and 100% with a scale of 0.01%</td></tr><tr><td>status</td><td>boolen</td><td>On/Off status of the fixture. True is on and false is off.</td></tr></table>
setPreset	Number	Set current motor position as the preset position:

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 Dongguan 2 Vietnam 3 Mexico 4 New York 5 Georgia 6 Los Angels
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to

		100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
schemVer	String	Version of the schematic used for the fixture.
pcbVer	String	Version of the PCB used for the fixture.
fwVer	String	Version of the firmware running on the fixture.
busVer	String	Version of the bus protocol running on the fixture.
upDownCircleTime	Number	The time that the up down motor running a circle[0-65535]
leftRightCircleTime	Number	The time that the left right motor running a circle[0-65535]
devAbility	Array	Optional: Specific functionality the fixture support bit0:CCT bit1:RGB bit2:Dim to warm bit3:Natural Dynamic bit4: device Dynamic bit5: sync dynamic

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”
        “devAbility”:[0,0]
}
}
```

ELV Single Color Dimmable Light

The ELV single color dimmable light is a virtual fixture. It can have the state and brightness controlled.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
online	Boolean	Read-only. True if the fixture is currently online and reachable, false otherwise.
mode	Number	Read-only. Enumeration of the fixture's current control mode.
colormode	String	Read-only. Current color mode of the fixture ("DIM", "CCT", or "RGB").
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.
findme	Boolean	On/Off findme function True Start findme , False Stop findme
requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        "status": true,
        "level": 7501,
        "findme":true
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** **Description**

		0 Linear dimming curve. 1 Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.

Example JSON configuration structure:

```
{
        "dimmingCurve": 0,
        "onRate": 200,
        "offRate": 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 Dongguan 2 Vietnam 3 Mexico 4 New York 5 Georgia 6 Los Angels
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
pcbVer	String	Version of the PCB used for the fixture.
fwVer	String	Version of the firmware running on the fixture.
busVer	String	Version of the bus protocol running on the fixture.

devAbility	Array	Optional: Specific functionality the fixture support bit0:CCT bit1:RGB bit2:Dim to warm bit3:Natural Dynamic bit4: device Dynamic bit5: sync dynamic

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”,
        “devAbility”:[0,0]
}
```

Wall Station

Wall Station is a virtual fixture. After associate button on it with the automation or group, it could trigger to execute the automation or control fixtures in the group.It also could
Dimming group or automation via press Up or Down button.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s state.

Format of the JSON control structure:

Name	Type	Description
online	Boolean	True If the wall station is online False If the wall Station is offline

Example JSON control structure:

```
{“online”: true}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** / **Description** 0 – Linear dimming curve. 1 – Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 – Dongguan 2 – Vietnam 3 – Mexico 4 – New York 5 – Georgia 6 – Los Angels
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
pcbVer	String	Version of the PCB used for the fixture.

fwVer	String	Version of the firmware running on the fixture.
busVer	String	Version of the bus protocol running on the fixture.

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”
}
```

24V Controller

The 24 controller allows the color temperature to be adjusted along with control of the state and brightness.

Control

When controlling the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture’s state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fixture. True is on and false is off.
level	Number	Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501.
colorTempLevel	Number	Value of the color temperature steps value between 1-7
MixColorTemp	Number	The value of Mix temperature It should not appear the same time.
DTWLevel	Number	Dim to warm level. Value of the light brightness between 0% and 100% with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000

		corresponds to 100%. To convert from percent to the input value multiply by 100. For example, 75.01% should be sent as 7501. The DCM should adjust the color temperature to meet the Dim To Warm curve
findme	Boolean	On/Off findme function True Start findme , False Stop findme
requestedMode	Number	0:Dimmable 1:Tunable white 2:RGB 3:HSV 4:Dim to warm 5:Natural 6:dynamic
funEnable	Boolean	Optional: enable or disable specific functionality. Ture: Enable False: Disable

Example JSON control structure:

```
{
        "status": true,
        "level": 7501,
          "findme":true,
         "colorTempLevel":100,
         "MixColorTemp":100
}
```

Configuration

When configuring the fixture, data should be provided in the following format. Data will also be returned in this format when reading the fixture's configuration.

Format of the JSON configuration structure:

Name	Type	Description
dimmingCurve	Number	Value representing an enumeration corresponding to the dimming curve that should be used. **Value** / **Description** 0 / Linear dimming curve. 1 / Logarithmic dimming curve.
onRate	Number	Rate at which the fixture transitions from the off to on state in milliseconds.
offRate	Number	Rate at which the fixture transitions from the on to off state in milliseconds.
colorTempCurve	Number	Value representing an enumeration corresponding to the curve that should be

		used. **Value** / **Description** 0 – Linear dimming curve. 1 – Logarithmic dimming curve.
dimToWarm	Boolean	True: enable the functionality of dim to warm False: disable the functionality of dim to warm
dimMode	Number	0: normal mode. CCT mode 1: single color temperature mode

Example JSON configuration structure:

```
{
        “dimmingCurve”: 0,
        “onRate”: 200,
        “offRate”: 200,
         “dimToWarm”:ture,
         “coloreTempCure”:0,
        “dimMode”:0
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration corresponding to the factory the fixture was produced and/or modified at. **Value** / **Description** 1 – Dongguan 2 – Vietnam 3 – Mexico 4 – New York 5 – Georgia 6 – Los Angeles
model	String	Model number of the fixture.
ledDriver	String	Model number of the LED driver.
currentOutput	Number	Rated current output level of the fixture in milliamps.
currentLevel	Number	Percentage of the customized current level of the fixture in percent with a scale of 0.01%. Possible values are between 0 and 10,000 where 10,000 corresponds to 100%. To convert from percent to the value multiply by 100. For example, 75.01% should be sent as 7501.
schemVer	String	Version of the schematic used for the fixture.

pcbVer	String	Version of the PCB used for the fixture.
fwVer	String	Version of the firmware running on the fixture.
busVer	String	Version of the bus protocol running on the fixture.
colorTempStepsTable	Array of Objects	colorStepsIndex: Index of the color steps1-7 MixColorTemp: The Mix color temperature of the steps.0-65535
minColorTemp	Number	Optional: Min color temperature of the fixture
maxColorTemp	Number	Optional:max color temperature of the fixture
devAbility	Array	Optional: Specific functionality the fixture support bit0:CCT bit1:RGB bit2:Dim to warm bit3:Natural Dynamic bit4: device Dynamic bit5: sync dynamic

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”,
“colorTempStepsTable”:
[
        {
        “colorStepsIndex”:1,
        “MixColorTemp”:1000
        }

        {
        “colorStepsIndex”:2,
        “MixColorTemp”:3000
        }

        ]
        “minColorTemp”:1000,
        “maxColorTemp”:6500,
        “devAbility”:[0,0]
```

```
}
```

Fan

The fan allows the motor speed and wind model to be adjusted along with control of the state and direction.

Control

When controlling the fab, data should be provided in the following format. Data will also be returned in this format when reading the fan's state.

Format of the JSON control structure:

Name	Type	Description
status	Boolean	On/Off status of the fan. True is on and false is off.
fanSpeed	Number (clamped range)	Value of the fan speed. There are 6 gears, from 1 to 6.
wind	Boolean	Enable or disable wind model
windSpeed	Number (clamped range)	The gear of the wind model.
fanDirection	Boolean	Direction of the fan, Ture: reserve False: forward.
findme	Boolean	On/Off findme function True Start findme , False Stop findme

Example JSON control structure:

```
{
        "status": true,
        "fanSpeed": 1,
        "fanDirection":true,
          "wind":false,
          "windSpeed":1
}
```

Configuration

TBD

Format of the JSON configuration structure:

Name	Type	Description

Example JSON configuration structure:

```
{
}
```

Detail

When reading the fixture, the detail data will be returned in the following format.

Format of the JSON detail structure:

Name	Type	Description
dateCode	String	Date that the device was manufactured. Will have the format: yyyy-mm-dd.
factory	Number	Value representing an enumeration

<table>
<tr><td></td><td></td><td>corresponding to the factory the fixture was produced and/or modified at.<table><tr><th>Value</th><th>Description</th></tr><tr><td>1</td><td>Dongguan</td></tr><tr><td>2</td><td>Vietnam</td></tr><tr><td>3</td><td>Mexico</td></tr><tr><td>4</td><td>New York</td></tr><tr><td>5</td><td>Georgia</td></tr><tr><td>6</td><td>Los Angeles</td></tr></table></td></tr>
<tr><td>model</td><td>String</td><td>Model number of the fan</td></tr>
<tr><td>ledDriver</td><td>String</td><td>Model number of motor</td></tr>
<tr><td>currentOutput</td><td>Number</td><td></td></tr>
<tr><td>currentLevel</td><td>Number</td><td></td></tr>
<tr><td>schemVer</td><td>String</td><td>Version of the schematic used for the fixture.</td></tr>
<tr><td>pcbVer</td><td>String</td><td>Version of the PCB used for the fixture.</td></tr>
<tr><td>fwVer</td><td>String</td><td>Version of the firmware running on the fixture.</td></tr>
<tr><td>busVer</td><td>String</td><td>Version of the bus protocol running on the fixture.</td></tr>
</table>

Example JSON configuration structure:

```
{
        “dateCode”: “2020-01-01”,
        “factory”: 1,
        “model”: “test”,
        “ledDriver”: “test”,
        “currentOutput”: 1000,
        “currentLevel”: 7510,
        “schemVer”: “0”,
        “pcbVer”: “0”,
        “fwVer”: “0.00”,
        “busVer”: “0.00”
        }
```

Appendix 2 –Status Codes

This lists the error codes in the system. These errors can be returning from the REST requests.

Error	Value
STRUT_SUCCESS	0
STRUT_GENERAL_FAILURE	-1
STRUT_INVALID_ADDR	-2
STRUT_INVALID_INPUT	-3
STRUT_INVALID_SIZE	-4
STRUT_STORAGE_ERROR	-5
STRUT_SCM_COMM_ERROR	-6
STRUT_INVALID_PARAM	-7
STRUT_INVALID_FIXTURE	-8
STRUT_INVALID_PROV	-9
STRUT_NOT_ENABLED	-10
STRUT_MAX_ENABLED	-11
STRUT_OVERFLOW	-12
STRUT_MAX_DEVICES	-13
STRUT_MAX_PAIRINGS	-14
STRUT_NOT_FOUND	-15
STRUT_ALREADY_EXISTS	-16
STRUT_ALREADY_PAIRED	-17
STRUT_NAME_TOO_LARGE	-18
STRUT_INVALID_ACTION	-19
STRUT_AUTOMATION_FULL	-20
STRUT_INVALID_FIXTURE_ADDR	-21
STRUT_GROUP_ADDR_INVALID	-22
STRUT_GROUP_INDEX_INVALID	-23
STRUT_SCENE_ADDR_INVALID	-24
STRUT_SCENE_INDEX_INVALID	-25
STRUT_MAX_INTEGRATIONS	-26
STRUT_OUT_OF_MEMORY	-27
STRUT_ADDRESS_EXISTS	-28
STRUT_NVS_GET_ERROR	-29
STRUT_NVS_SET_ERROR	-30
STRUT_NVS_ERASE_ERROR	-31
STRUT_SIZE_MISMATCH	-32
STRUT_INVALID_TYPE	-33

Error	Value
STRUT_CREATE_JSON_ERROR	-34
STRUT_MISS_TYPE	-35
STRUT_MISS_FIXTURES	-36
STRUT_MISS_GROUPS	-37
STRUT_MISS_OCCSENSOR	-38
STRUT_AUTOM_RES_FAIL	-39
STRUT_CANT_REMOVE_INTEGRATED_DEVICE	-40
STRUT_MAX_GROUPS	-41
STRUT_NO_MAPPING	-42
STRUT_MISSING_NAME	-43
STRUT_MISSING_REQUIRED_PARAM	-44
STRUT_MISS_CONFIG	-48
STRUT_MISS_PAYLOAD	-50
STRUT_MISS_REQUEST_ID	-51
STRUT_MISS_TIMESTAMP	-52
STRUT_MISS_PAYLOAD	-50
STRUT_MISS_REQUEST_ID	-51
STRUT_MISS_TIMESTAMP	-52
STRUT_MISS_ACTION	-53
STRUT_MISS_STATE	-54
STRUT_MISS_TUNE	-55
Network Errors	
STRUT_SCAN_IN_PROGRESS	-60
STRUT_SCAN_NOT_STARTED	-61
STRUT_WIFI_OFF	-62
STRUT_AWS_NOT_CONNECTED	-63
STRUT_AWS_CLOUD_FAILURE	-64
STRUT_AWS_TIMED_OUT	-65
STRUT_AWS_ALREADY_COMMISSIONED	-66
STRUT_AWS_NOT_COMMISSIONED	-67
Batch OTA	
STRUT_CHECKSUM_ERROR	-70
STRUT_GET_IMAGE_ERROR	-71
STRUT_WIRTE_IMAGE_ERROR	-72
STRUT_SUBTYPE_ERROR	-73
DMX	
STRUT_MISS_ENABLED	-80
STRUT_MISS_MAPPING	-81
STRUT_MISS_THING	-82
Basic OTA	

Error	Value
STRUT_OTA_UPDATE_NOT_STARTED	-102
STRUT_OTA_VALIDATION_FAILED	-103
STRUT_OTA_END_FAILED	-104
STRUT_OTA_SAME_IMAGE	-105
STRUT_OTA_SET_BOOT_FAILED	-106
STRUT_OTA_UPDATE_IN_PROGRESS	-107
STRUT_OTA_PARTITION_FAILED	-108
STRUT_OTA_IMAGE_SIZE	-109
STRUT_OTA_BEGIN_FAILED	-110
STRUT_OTA_INVALID_PARAMS	-111
STRUT_OTA_WRITE_FAILED	-112